

Sistema de detección de colores con OpenCV

1. Objetivo

Desarrollar un sistema de visión por computadora capaz de detectar objetos de distintos colores en tiempo real usando una cámara, identificando su posición respecto al centro de la imagen. El sistema permite reconocer objetos **amarillos**, **verdes** y **rojos**, además de indicar si cada objeto se encuentra a la izquierda, derecha o centrado.

2. Descripción general

El programa captura video en tiempo real desde una cámara conectada al dispositivo y procesa cada cuadro usando el espacio de color **HSV**. Para cada color definido:

- se genera una máscara binaria,
- se elimina ruido,
- se detectan contornos,
- se filtran objetos pequeños,
- se obtiene la posición del objeto,
- se dibuja un rectángulo y etiquetas sobre la imagen.

Además, el sistema calcula el error horizontal y vertical entre el centro del objeto y el centro del frame.

3. Requisitos

Hardware

- Computadora, Jetson o Raspberry Pi con cámara USB o cámara compatible.
- Cámara funcional conectada al equipo.

Software

- Python 3
- OpenCV
- NumPy
- Terminal de Linux

4. Creación del entorno de trabajo

carpeta para el proyecto.

```
mkdir deteccion_colores
cd deteccion_colores
```

5. Crear el entorno virtual

Para evitar conflictos con otras librerías del sistema, se crea un entorno virtual.

```
python3 -m venv env
```

6. Activar el entorno virtual

activarlo con:

```
source env/bin/activate
```

7. Instalar dependencias

Con el entorno activado, instalar las librerías:

```
pip install opencv-python numpy
```

8. Programa

```
nano color_detection_test.py
```

```
import cv2
```

```
import numpy as np
```

```
print("Iniciando detección de colores...")
```

```
cap = cv2.VideoCapture(2)
```

```

if not cap.isOpened():

    print("No se pudo abrir la cámara")

    exit()


cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)


threshold_center = 40

min_area = 500


color_ranges = {

    "Azul": [ # detecta amarillo

        (np.array([20, 100, 100]), np.array([30, 255, 255]))

    ],

    "Verde": [

        (np.array([35, 80, 50]), np.array([85, 255, 255]))

    ],

    "Rojo": [

        (np.array([0, 120, 70]), np.array([10, 255, 255])),

        (np.array([170, 120, 70]), np.array([180, 255, 255]))

    ]

}


draw_colors = {

    "Azul": (0, 255, 255),

    "Verde": (0, 255, 0),

```

```
"Rojo": (0, 0, 255)
}

print("Cámara abierta. Presiona ESC para salir.")

while True:

    ret, frame = cap.read()

    if not ret:

        print("No se pudo leer frame")

        break

    hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)

    frame_h, frame_w = frame.shape[:2]

    frame_center_x = frame_w // 2

    frame_center_y = frame_h // 2

    cv2.circle(frame, (frame_center_x, frame_center_y), 6, (255, 255, 255), -1)

    cv2.putText(frame, "Centro", (frame_center_x + 10, frame_center_y - 10),

                cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255, 255, 255), 1)

    combined_mask = np.zeros((frame_h, frame_w), dtype=np.uint8)

    total_objects = 0

    for color_name, ranges in color_ranges.items():

        mask = np.zeros((frame_h, frame_w), dtype=np.uint8)
```

for lower, upper in ranges:

```
partial_mask = cv2.inRange(hsv, lower, upper)
```

```
mask = cv2.bitwise_or(mask, partial_mask)
```

```
kernel = np.ones((5, 5), np.uint8)
```

```
mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel)
```

```
mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, kernel)
```

```
combined_mask = cv2.bitwise_or(combined_mask, mask)
```

```
contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL,  
cv2.CHAIN_APPROX_SIMPLE)
```

```
obj_index = 0
```

for cnt in contours:

```
area = cv2.contourArea(cnt)
```

```
if area < min_area:
```

```
    continue
```

```
obj_index += 1
```

```
total_objects += 1
```

```
x, y, w, h = cv2.boundingRect(cnt)
```

```
cx = x + w // 2
```

```
cy = y + h // 2
```

```
error_x = cx - frame_center_x
```

```
error_y = cy - frame_center_y
```

```
if error_x > threshold_center:
```

```
    estado = "Derecha"
```

```
elif error_x < -threshold_center:
```

```
    estado = "Izquierda"
```

```
else:
```

```
    estado = "Centrado"
```

```
draw_color = draw_colors[color_name]
```

```
cv2.rectangle(frame, (x, y), (x + w, y + h), draw_color, 2)
```

```
cv2.circle(frame, (cx, cy), 5, (0, 255, 255), -1)
```

```
cv2.line(frame, (frame_center_x, frame_center_y), (cx, cy), draw_color, 2)
```

```
label_1 = f"{color_name}_{obj_index}"
```

```
label_2 = f"err_x:{error_x} err_y:{error_y}"
```

```
label_3 = f"{estado}"
```

```
cv2.putText(frame, label_1, (x, y - 35),
```

```
            cv2.FONT_HERSHEY_SIMPLEX, 0.6, draw_color, 2)
```

```
cv2.putText(frame, label_2, (x, y - 15),
```

```
            cv2.FONT_HERSHEY_SIMPLEX, 0.5, draw_color, 2)
```

```
cv2.putText(frame, label_3, (x, y + h + 20),  
            cv2.FONT_HERSHEY_SIMPLEX, 0.6, draw_color, 2)
```

```
cv2.putText(frame, f"Objetos detectados: {total_objects}", (10, 25),  
            cv2.FONT_HERSHEY_SIMPLEX, 0.7, (255, 255, 255), 2)
```

```
cv2.putText(frame, "Blanco = centro de imagen", (10, 50),  
            cv2.FONT_HERSHEY_SIMPLEX, 0.55, (255, 255, 255), 1)
```

```
cv2.imshow("Video", frame)  
cv2.imshow("Mascara combinada", combined_mask)
```

```
key = cv2.waitKey(1) & 0xFF  
if key == 27:  
    break
```

```
cap.release()  
cv2.destroyAllWindows()  
print("Programa terminado")
```

y ejecutar:

variables de entorno para OpenCV:

```
export QT_QPA_PLATFORM=xcb  
python3 color_detection_test.py
```

** Si cámara no abre, checar puerto ls /dev/video* de la camara.
